Table des matières

Cours 04 — Lire une base et une API 1
Objectifs 1
1. Tirer ou recevoir : MQTT, REST et ODBC 1
2. Lire une base par ODBC 2
3. La capture de changements 3
4. Tirer une API REST 5
5. Lire une source qu'on tire, en pratique 6
À retenir 7
Exercices 7
Annexe A — Le détail des étapes du labo 8

Cours 04 — Lire une base et une API

Théorie · Module 4/12 — Formation Data Engineering · intermédiaire

Où vous êtes : source → **ingestion** → lac → entrepôt → restitution.

Objectifs

- Situer une source qui pousse et une source qu'on tire, et ce que chacune laisse à votre charge.
- Lire une base relationnelle par ODBC sans la mettre en danger.
- Capter les changements d'une base par requête, et connaître les limites de la méthode.
- Tirer une API REST paginée avec des reprises, et prouver qu'on a tout reçu.

---

1. Tirer ou recevoir : MQTT, REST et ODBC

Le Cours 03 a chargé le dépôt de fichiers, que la source écrit et que vous ramassez. Les deux autres voies se tirent : la base, par un pilote, et l'API, page par page. MQTT, la voie de la télémétrie en direct, suit une autre logique (Cours 09).

© 2026 iAlexMG. Tous droits réservés.

	MQTT	REST	ODBC
Qui prend l'initiative	la source publie, l'abonné reçoit	le client demande	le client interroge
Ce qui circule	des messages courts, rangés par sujet	des documents, souvent en JSON, sur HTTP	des lignes de tables, en réponse à du SQL
Garantie de livraison	trois niveaux de qualité de service, dont « au moins une fois », qui produit des doublons	aucune au-delà de la réponse : le client réessaie	celle de la base : une requête voit un état cohérent
Usage type	des capteurs et des véhicules qui publient en continu	des services ouverts, des applications infonuagiques	les bases d'entreprise, l'ERP
Chez Boréal	la télémétrie et l'usine en direct (Cours 09)	l'API d'ECCC	la base de l'ERP

À retenir — tirer ou recevoir. Un système qui pousse (MQTT) livre ce qu'il a, quand il l'a : il faut être là pour le recevoir. Un système qu'on tire (REST, ODBC) attend la demande : c'est vous qui savez quoi demander, et quand. Presque tous les pièges d'une ingestion viennent de ce partage des responsabilités.

2. Lire une base par ODBC

Pour lire une base, un programme doit parler sa langue. Chaque moteur (PostgreSQL, SQL Server, Oracle, SAP HANA) a la sienne : sa façon d'échanger requêtes et lignes sur le réseau, qu'on appelle son protocole. ODBC (*Open Database Connectivity*) évite au programme de les apprendre toutes : le programme parle ODBC, toujours de la même façon, et un pilote, propre à chaque moteur, traduit.

Analogie — l'adaptateur de voyage. L'appareil garde la même fiche dans tous les pays ; c'est l'adaptateur qui change. Le script garde les mêmes appels pour toutes les bases ; c'est le pilote qui change.

Le chemin d'une requête, dans le labo :

Maillon	Ce qu'il fait	Dans le labo
le script	ouvre une connexion, envoie du SQL, lit les lignes	`extraire_erp_odbc.py`, avec le paquet `pyodbc`
le gestionnaire de pilotes	trouve le pilote que le script demande	celui de Windows
le pilote	traduit les appels ODBC dans la langue de la base	psqlODBC, le pilote de PostgreSQL
la base	exécute la requête et renvoie les lignes	PostgreSQL 18, dans son conteneur, sur le port 5433

© 2026 iAlexMG. Tous droits réservés.

- Ce que ça change : le script ne contient rien de propre à PostgreSQL, sauf le nom du pilote dans sa chaîne de connexion. Pour lire SQL Server ou Oracle, on change le pilote et la chaîne ; les appels restent les mêmes. C'est ce qui permet à un même outil d'entreprise de lire toutes les bases.
- La chaîne de connexion dit où aller et qui entre : `DRIVER={PostgreSQL Unicode(x64)};SERVER=localhost;PORT=5433;DATABASE=erp_boreal;UID=postgres;PWD=…`.
- En Python, `pyodbc` s'emploie comme `duckdb` : `connect`, `cursor`, `execute`, `fetchmany`. Les deux suivent la même interface standard des bases de données de Python, DB-API 2.0.

Lire la base d'un ERP demande des égards : pendant que le script lit, les employés saisissent des commandes dans la même base. Une longue lecture leur prend des disques et de la mémoire, et, tant qu'elle dure, PostgreSQL doit garder l'ancienne version de chaque ligne modifiée entre-temps.

Usage	**Ce qu'il évite**
extraire hors des heures de pointe, ou sur une réplique en lecture	de ralentir ceux qui saisissent les commandes
un compte qui n'a que le droit de lire	qu'une erreur de script modifie la base
lire par paquets (`fetchmany`)	de tenir une table entière en mémoire
ne lire que les lignes changées (section 3) et les colonnes utiles	de relire chaque nuit ce qui n'a pas bougé

- La base tourne dans Ubuntu, et Windows la joint pourtant par `localhost:5433` : WSL relaie vers Windows les ports ouverts dans Ubuntu. Le 5432 reste libre pour un PostgreSQL déjà installé sur le poste.

Piège — Ubuntu qui s'arrête dans votre dos. WSL arrête Ubuntu peu après la fermeture du dernier terminal Ubuntu, et Docker s'arrête avec elle : le script Windows qui voulait lire la base reçoit une connexion refusée. Le délai se règle par `instanceIdleTimeout`, dans `%UserProfile%\.wslconfig` : c'est l'étape 5 du guide d'installation du parcours. À défaut, gardez un terminal Ubuntu ouvert pendant que vous travaillez avec la base.

3. La capture de changements

Relire toute une base chaque nuit ne tient pas longtemps. Capter les changements (*change data capture*), c'est n'extraire que ce qui a été créé, modifié ou supprimé depuis la dernière fois. Trois méthodes, de la plus simple à la plus complète.

3.1 Par requête, sur un filigrane

On retient la plus haute date de modification déjà extraite, le filigrane (*watermark*), et l'on demande la suite. Une commande de l'ERP porte deux dates en texte : `ERDAT`, sa création, et `AEDAT`, sa dernière modification, qui vaut `00000000` tant qu'elle n'a jamais été modifiée.

© 2026 iAlexMG. Tous droits réservés.

Filtre, pour décembre 2025	Commandes
modifiées en décembre (`AEDAT >= '20251201'`)	37
créées en décembre et jamais modifiées	390
créées avant décembre, modifiées en décembre	9
créées ou modifiées en décembre (`ERDAT >= '20251201' OR AEDAT >= '20251201'`)	427

- Un filigrane posé sur `AEDAT` seul raterait 390 commandes. Posé sur les deux dates, il rend les 427 commandes de l'extrait de décembre, ligne pour ligne : c'est ainsi que l'ERP fabrique ses extraits mensuels.
- Le filigrane a besoin d'un index. La base en porte un sur `AEDAT` : un filtre sur cette date seule passe par lui ; le filtre complet, avec son `OR` sur `ERDAT` qui n'a pas d'index, lit la table entière. Sur 25 483 commandes, c'est instantané ; sur des centaines de millions de lignes, c'est l'extraction qui bloque la nuit.

Angle mort	Ce qui se passe	Parade
les suppressions	une ligne supprimée ne revient dans aucune requête	une suppression logique, comme l'indicateur `LOEVM` des concessionnaires fermés ; sinon, une autre méthode
une date au jour près	`AEDAT` n'a pas d'heure : une ligne modifiée plus tard le même jour tombe entre deux extractions	relire le jour du filigrane (`>=` plutôt que `>`), et compter sur l'idempotence
les transactions longues et les horloges	une ligne validée après l'extraction, avec une date antérieure au filigrane, passe entre les mailles	une marge de sécurité : recommencer un peu avant le filigrane
un programme qui ne tient pas la date à jour	un traitement en lot qui modifie sans toucher `AEDAT`	vérifier la règle avec l'équipe de l'ERP, et le prouver par un contrôle

3.2 Par les documents de modification

- Certains logiciels tiennent eux-mêmes le registre de ce qui change. Dans SAP, les documents de modification notent, pour les objets configurés, qui a modifié quel champ, quand, par quelle transaction, avec l'ancienne et la nouvelle valeur : `CDHDR` en porte l'en-tête, `CDPOS` le détail.
- Chez Boréal, seules les fiches des concessionnaires sont suivies : 53 changements, dont 49 changements de palier (`KUKLA`) le 2024-07-01, par le programme de révision `ZPALIER`, et 4 fermetures (`LOEVM`), saisies à la main par `XD02`.

© 2026 iAlexMG. Tous droits réservés.

- Leur force : garder chaque changement, même quand la fiche a changé encore depuis. Le palier d'avant la révision n'existe plus dans `KNA1`, mais `CDPOS` le garde. Leur limite : ne couvrir que ce que l'entreprise a choisi de suivre.

3.3 Par le journal de la base

- Toute base transactionnelle écrit chaque changement dans son journal avant de l'appliquer : le *WAL* de PostgreSQL, le *binlog* de MySQL. Un outil de capture, comme Debezium, lit ce journal et en fait un flux d'événements : chaque insertion, modification et suppression, dans l'ordre, sans interroger les tables.

- C'est la méthode la plus complète et la plus légère pour la base, mais elle demande l'accès au journal, un connecteur à exploiter et un format d'événements à comprendre. La synthèse Comptoir Aurore rejoue un tel journal, de l'instantané initial jusqu'à la `RAW`.

Méthode	Voit les suppressions	Charge sur la base	Ce qu'elle demande
Filigrane	non	une requête, et un index	une date de modification fiable
Documents de modification	selon la configuration	faible	un logiciel qui les tient
Journal de la base	oui	presque nulle	l'accès au journal et un connecteur

4. Tirer une API REST

Une API REST répond à des requêtes HTTP : le client demande une adresse, avec ses paramètres ; le serveur répond par un code et un document. L'API d'ECCC, `api.weather.gc.ca`, suit la norme OGC API Features : des collections (`climate-daily`, `climate-hourly`, `climate-stations`), des entités filtrées par propriété (`CLIMATE_IDENTIFIER=7060400`) ou par période (`datetime`), et trois formats de réponse, JSON, CSV et HTML.

Élément de la pagination	Ce qu'il dit	Pour Bagotville, d'octobre à décembre 2025, pages de 50
`limit`, `offset`	la taille de la page, et le rang de la première entité	50, puis 0 et 50
`numberMatched`	le nombre d'entités qui répondent à la requête	92
`numberReturned`	le nombre d'entités de cette page	50, puis 42
le lien `next`	l'adresse de la page suivante, absente sur la dernière	présent sur la première page seulement

- Le serveur plafonne la taille d'une page. ECCC l'annonce à 10 000 entités : une demande de 15 000 observations horaires de Bagotville rend `numberReturned` à 10 000, sur 37 068 annoncées. Un client qui ne lirait que la première réponse croirait tout avoir : on compare donc, à la fin, les entités reçues à `numberMatched`.

© 2026 iAlexMG. Tous droits réservés.

Code HTTP	Sens	Réessayer
200	réponse normale	sans objet
400	requête fausse ; ECCC répond `InvalidParameterValue` pour un `limit` qui n'est pas un nombre	non : la même requête échouera encore
404	ressource absente ; ECCC répond `NotFound` pour une collection inconnue	non
429	trop de requêtes : le serveur demande de ralentir, parfois en indiquant le délai par l'en-tête `Retry-After`	oui, après le délai
5xx	panne du serveur	oui, après un délai
aucun code	coupure du réseau, délai dépassé	oui, après un délai

- Le délai double à chaque échec (2, 4, puis 8 secondes dans le labo) : c'est la temporisation exponentielle, qui laisse au serveur le temps de se remettre au lieu de l'assaillir. Le nombre d'essais reste borné, et la dernière erreur remonte. Le Cours 05 fait de ces reprises un outil commun.
- ECCC n'annonce pas de limite de débit : raison de plus pour rester poli, avec des pages assez grandes et des requêtes l'une après l'autre.
- La météo de la `RAW` vient d'un instantané figé de cette même API, pour que tout le monde obtienne les mêmes chiffres. Comparées en texte, l'API et l'instantané semblent différer : `6.0` d'un côté, `6` de l'autre. Comparées en nombres, elles concordent.

Piège — comparer des nombres en texte. `'6.0'` et `'6'` sont deux chaînes différentes et le même nombre. Une comparaison faite dans le mauvais type invente des écarts, ou en cache d'autres. Comparez toujours deux valeurs dans le type de ce qu'elles représentent.

5. Lire une source qu'on tire, en pratique

Une source qu'on tire ne dit rien d'elle-même : l'ingénieur de données pose toujours les mêmes questions, et chacune a sa preuve.

Question	Ce qu'on vérifie	Chez Boréal (labo)
La source répond-elle ?	le service et son port	`boreal_erp` à l'état `Up`, sur le port 5433 (avant de commencer)
Que coûte ma requête à la base ?	son plan	par l'index sur `AEDAT`, contre toute la table avec le `OR` (1.3, 1.4)

© 2026 iAlexMG. Tous droits réservés.

Question	Ce qu'on vérifie	Chez Boréal (labo)
Ma capture voit-elle tout ?	le filigrane contre l'extrait	427 et 427, 0 écart ; 390 ratées par `AEDAT` seul (1.2, 3.3, 3.4)
Deux voies racontent-elles la même histoire ?	la base contre la `RAW` reconstruite	9 tables, 0 écart dans les deux sens (2.6)
Ai-je tout reçu de l'API ?	les entités reçues contre les annoncées	92 sur 92, en 2 pages (4.2)
Relancer double-t-il ?	une seconde extraction	le même fichier, 92 jours (4.3)
Mes écarts sont-ils réels ?	la comparaison dans le bon type	50 en texte, 0 en nombres (4.5, 4.7)
Le secret reste-t-il dehors ?	ce que Git ignore	`.gitignore:2:.env` (5)

Ces preuves se font à la main ici. Le pipeline du Cours 05 les garde, et le Cours 11 en fait des contrats.

À retenir

- Une source qui pousse livre ce qu'elle a ; une source qu'on tire attend la demande. Presque tous les pièges d'une ingestion viennent de ce partage des responsabilités.
- ODBC fait du pilote un contrat. On lit une base de production par paquets, hors des pointes, avec un compte en lecture.
- Un filigrane capte les créations et les modifications, pas les suppressions ; il demande une date fiable et un index. Les documents de modification et le journal de la base vont plus loin.
- Une API se lit jusqu'à la dernière page, avec des reprises bornées sur les erreurs passagères seulement, et l'on compte ce qu'on a reçu.
- Deux valeurs se comparent dans le type de ce qu'elles représentent.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_04_DE.md`. Faites-les après le labo : plusieurs énoncés supposent que la base de l'ERP tourne dans son conteneur.

© 2026 iAlexMG. Tous droits réservés.

Annexe A — Le détail des étapes du labo

A.0 Avant de commencer : recevoir la plateforme

Notions : section 2, la base dans son conteneur.

- Chaque fenêtre garde son dossier du début à la fin. `wsl` ouvre Ubuntu dans le dossier courant : la fenêtre 2 s'ouvre donc dans `plateforme`, où `docker compose` trouve son fichier. Les scripts Python calculent eux-mêmes le dossier du projet : ils se lancent par leur chemin, depuis `Projet_Boreal`, et Git se lance dans le dépôt par `git -C 01_de`.
- Dans l'Anaconda Prompt, un commentaire commence par `REM` ; dans Ubuntu, par `#`. Un `#` en fin de ligne, dans l'Anaconda Prompt, partirait vers la commande comme un argument de plus.
- La plateforme ne s'écrit pas : elle se reçoit. Ses fichiers vivent dans `_univers/plateforme/` : le `docker-compose.yml`, qui décrit ses services (la base de l'ERP, Airflow et ceux des cours suivants), le `Dockerfile.airflow`, qui construit l'image d'Airflow, et le modèle de son `.env`. En entreprise, c'est l'équipe qui exploite la plateforme qui les tient : on les copie sans les modifier. Ce cours emploie le service de l'ERP, le Cours 06 ceux d'Airflow.
- `docker compose` lit tout seul le `.env` voisin du `docker-compose.yml` : aucune commande ne le nomme, et c'est pourquoi ils vivent dans le même dossier.
- Le mot de passe se tient aux lettres et aux chiffres : un `#` ferait passer la suite de la ligne pour un commentaire, un `$` serait pris pour une variable, et des guillemets entreraient dans la valeur.
- PostgreSQL applique `POSTGRES_PASSWORD` quand il initialise sa base, au tout premier démarrage du volume, et jamais ensuite. Changer le `.env` après coup ne change donc pas le serveur : tout marche encore par `docker compose exec`, qui entre par le conteneur sans mot de passe, mais la lecture par ODBC de l'étape 2 échoue. Le dépannage du labo donne les deux sorties.
- `docker compose up -d erp` télécharge l'image, puis crée un réseau, le volume `boreal_erp_donnees` et le conteneur `boreal_erp`. `restart: unless-stopped` le fait repartir, avec ses données, quand Ubuntu redémarre. Le conteneur est `Up` avant que PostgreSQL accepte les connexions : d'où les quelques secondes d'attente.
- `docker compose exec` lance une commande dans le conteneur du service `erp` ; `-w /erp` la lance depuis le dossier monté des sources de l'ERP, où `charger_erp.sql` trouve ses CSV. Au second passage, le script annonce qu'il supprime le schéma (`NOTICE: drop cascades to 9 other objects`) avant de le recréer : il se rejoue autant de fois qu'on veut, et rend toujours les mêmes comptes.

A.1 Étape 1 : interroger la base de l'ERP

Labo : `Labo_04_DE.md`, étape 1.

Notions : section 3, la capture de changements ; section 2, lire une base avec des égards.

© 2026 iAlexMG. Tous droits réservés.

- 1.1. `psql` est le client de PostgreSQL, lancé ici dans le conteneur. Trois invites se ressemblent, et chacune parle sa langue : `erp_boreal=#`, c'est PostgreSQL ; `D`, c'est DuckDB ; une ligne qui finit par `$`, c'est Ubuntu, qui ne connaît pas le SQL. Dans `psql`, `--` ouvre un commentaire, et rien ne part avant le `;`.
- 1.2. Le filigrane de décembre prend les deux dates : 427 commandes.
- 1.3. `EXPLAIN` montre le plan que PostgreSQL choisit, sans exécuter la requête. Il le choisit d'après ses statistiques sur la table. Juste après le chargement, il n'en a pas encore : il estime 8 494 lignes et prend un `Bitmap Index Scan`. L'analyse automatique passe dans la minute ; il en estime alors 36 et prend un `Index Scan`. Les deux passent par l'index `ix_vbak_aedat`. Sur la base d'un ERP, on ne lance pas soi-même `ANALYZE` : c'est l'affaire de l'équipe qui l'exploite. L'index et la lecture d'un plan sont au Cours 10 de la formation SQL avancé, sections 2 et 3 ; ici, on ne regarde que si la requête passe par l'index.
- 1.4. Le `OR` sur `ERDAT`, qui n'a pas d'index, oblige à lire toute la table (`Seq Scan`). Sur 25 483 commandes, c'est instantané ; sur une vraie base de production, c'est la requête qu'on n'envoie pas à midi.
- 1.5. Les 49 changements de palier du 2024-07-01 viennent du programme de révision `ZPALIER` ; les quatre fermetures de concessionnaires ont été saisies à la main par `XD02`.
- 1.6. `\q` quitte `psql` ; la base, elle, continue de tourner.

A.2 Étape 2 : l'extraction par ODBC et sa preuve

Labo : `Labo_04_DE.md`, étape 2.

Notions : section 2, lire une base par ODBC ; section 1, tirer une source.

- 2.1. Le dépôt reçoit son propre `.env`, distinct de celui de la plateforme : celui de la plateforme sert à `docker compose`, qui a initialisé la base ; celui-ci sert au code, qui s'y connecte. `findstr /B` ne recopie que la ligne qui commence par `ERP_MOT_DE_PASSE=` : le mot de passe ne se tape qu'une fois, et les autres lignes de la plateforme restent où elles sont. `>` écrit la sortie dans le fichier, et le crée au besoin.
- 2.2. Le `.env` ne quitte jamais le poste : personne d'autre ne peut savoir de quelles variables le projet a besoin. Le `.env.exemple`, versionné, porte leurs noms avec des valeurs bidon ; qui clone le dépôt le copie en `.env`, met ses valeurs et démarre. La convention se retrouve partout, de dbt à Airflow : un dépôt sans son `.env.exemple` est un dépôt qu'on ne sait pas lancer.
- 2.3. Le script garde le premier pilote dont le nom commence par `PostgreSQL Unicode`, soit `PostgreSQL Unicode(x64)` : Unicode parce que les libellés de l'ERP portent des accents, 64 bits parce que le Python de `formations` l'est. Ce que fait chaque morceau :
  - `lire_env` lit le mot de passe dans `01_de/.env` : il n'est écrit ni dans le code, ni dans le dépôt ;
  - la chaîne de connexion nomme le pilote, le serveur, le port 5433, la base, l'utilisateur et le mot de passe ;
  - `fetchmany(PAQUET)` lit 10 000 lignes à la fois : la mémoire du script ne dépend pas de la taille de la table ;

© 2026 iAlexMG. Tous droits réservés.

    - le CSV temporaire garde le texte reçu, que `COPY` range en Parquet compressé ; le CSV part ensuite.

- 2.4. Neuf fichiers Parquet dans `plateforme/lac/bronze/erp_base/`, un par table.
- 2.5. Le contrôle garde, pour chaque clé de chaque table, la version du lot le plus récent (`QUALIFY ROW_NUMBER() … = 1`) : c'est la règle du Cours 03, section 2, qui retrouve l'état d'une base dans des extraits incrémentaux. Puis il compare cet état à la base, dans les deux sens (`EXCEPT`). `FROM 'fichier.parquet'` est la forme courte de `SELECT * FROM`. La numérotation par fenêtre et `QUALIFY` sont au Cours 04 de la formation SQL avancé, sections 3 et 7, `EXCEPT` et le rapprochement dans les deux sens à son Cours 07, section 6, et `SELECT * EXCLUDE (…)`, qui écarte les colonnes de lignage, à son Cours 02, section 1.
- 2.6. Neuf tables, 0 écart dans les deux sens : deux voies d'extraction indépendantes racontent la même histoire. Les champs vides concordent aussi : la base stocke des chaînes vides, le CSV les transporte en champs vides, et DuckDB les lit en `NULL` des deux côtés. La reconstruction est une preuve, pas une livraison : la `RAW` garde toutes les versions, et le `STAGING` d'Analytics Engineering refera ce travail pour de bon.

A.3 Étape 3 : le filigrane contre l'extrait

Labo : `Labo_04_DE.md`, étape 3.

Notions : section 3, le filigrane et ses angles morts.

- 3.1. `READ_ONLY` : on regarde. Le contrôle de 2.6 a relâché l'entrepôt en finissant.
- 3.2. L'extrait mensuel de décembre compte 427 en-têtes, autant que le filigrane posé sur la base.
- 3.3. L'état reconstruit au 2025-12-31 se découpe en 37 commandes modifiées en décembre, 390 créées en décembre et jamais modifiées, dont `AEDAT` vaut `00000000`, et 9 anciennes modifiées. Un filigrane sur `AEDAT` seul raterait les 390.
- 3.4. Le filigrane et l'extrait donnent exactement les mêmes 427 lignes : l'extrait de l'ERP n'est rien d'autre que cette requête, lancée la nuit qui suit la fin du mois.

A.4 Étape 4 : l'API d'ECCC, page par page

Labo : `Labo_04_DE.md`, étape 4.

Notions : section 4, tirer une API REST ; section 1, une source qu'on tire.

- 4.1. Ce que fait chaque morceau du script :
    - `parametres` : la station de Bagotville A, la période, le tri par date, et des pages de 50 jours pour voir la pagination (l'API en accepte jusqu'à 10 000) ;
    - `lire_page` : une requête, et ses reprises, 2, 4, puis 8 secondes après une coupure, un 429 ou une erreur 5xx ; aucune reprise après une autre erreur 4xx ;

© 2026 iAlexMG. Tous droits réservés.

    - le lien `next` : la page suivante, telle que l'API la construit ; la boucle s'arrête quand il n'y en a plus ;
    - `recus != annonces` : la preuve qu'on a tout reçu, la somme des `numberReturned` contre `numberMatched`.

- 4.2. 92 jours en deux pages, 50 puis 42 : la dernière page n'a pas de lien `next`.
- 4.3. Le script réécrit le même fichier : relancer ne double rien, l'extraction est idempotente.
- 4.4. Le JSON de l'API porte des nombres, que DuckDB lit en nombres ; l'instantané de la `RAW` les garde en texte.
- 4.5. Comparées en texte, 50 valeurs semblent différer : 5 températures moyennes et 45 précipitations.
- 4.6. `6.0` contre `6` : deux textes différents, un seul nombre.
- 4.7. Comparées en nombres, l'API et l'instantané concordent sur les 92 jours.
- 4.8. L'entrepôt se relâche avant tout script qui écrit.

A.5 Étape 5 : versionner

Labo : `Labo_04_DE.md`, étape 5.

Notions : aucune section de ce cours ; le `.gitignore` et sa ligne `.env` viennent du Cours 01, annexe A.1.

- `git -C 01_de` lance Git comme s'il était dans `01_de`, sans que la fenêtre quitte `Projet_Boreal` : les chemins qui suivent partent du dépôt.
- `git add` nomme les quatre fichiers : les deux scripts, le contrôle et `.env.exemple`.
- `git check-ignore -v .env` dit quelle règle tient le `.env` dehors : la deuxième ligne du `.gitignore`.
- Le `docker-compose.yml` n'est pas dans ce dépôt : il appartient à la plateforme, que l'équipe qui l'exploite tient.

En entreprise, les mêmes gestes changent d'outil :

Geste du labo	**En entreprise**
Lire une base par un pilote (2)	un pilote, vers une réplique en lecture, par un outil d'intégration ; dans Databricks, `spark.read.format("jdbc")`
Capter les changements (1, 3)	le journal de la base, lu par un outil de capture comme Debezium, plutôt qu'un filigrane

© 2026 iAlexMG. Tous droits réservés.

Geste du labo	En entreprise
Tirer une API (4)	le même code, lancé par l'orchestrateur (Cours 06), qui garde ses reprises et son suivi
Garder le mot de passe hors du code (2)	un coffre à secrets : AWS Secrets Manager, Azure Key Vault, les secrets de Databricks

© 2026 iAlexMG. Tous droits réservés.